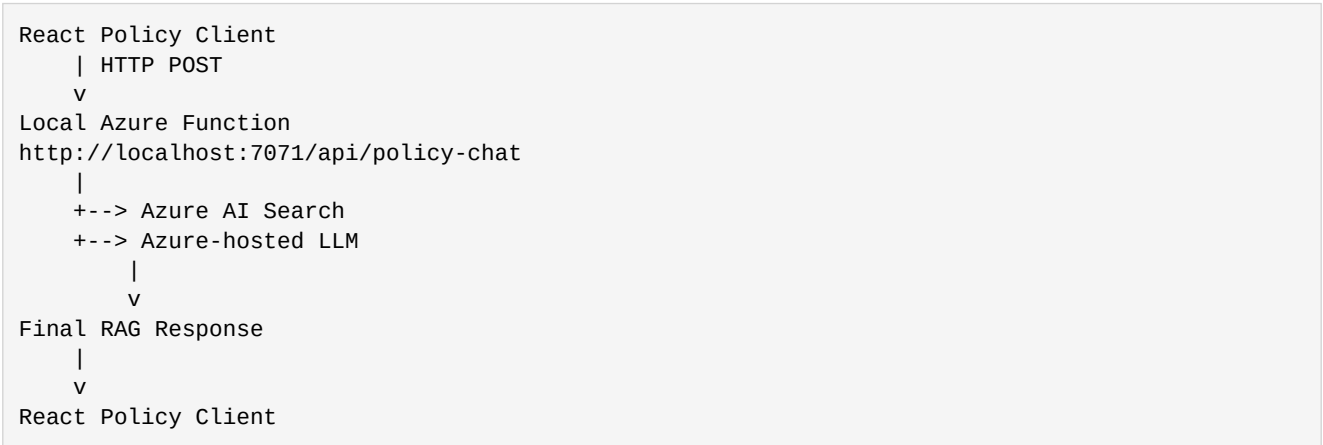


Policy Assistant Integration Recommendation

Recommendation: Connect the React Policy client directly to the local Policy Function App first. Add Azure API Management (API Gateway) after the end-to-end application flow is working.

Recommended Immediate Architecture

The next milestone should prove the complete application flow:



You have already proven that the Azure Function can receive a query and return an LLM response. The next logical step is to prove **React -> Function -> Search -> LLM -> Function -> React**.

Why Not Add API Management Yet?

Adding API Management now introduces another troubleshooting layer while the front-end integration is still being completed. If React cannot receive a response, the problem could be in CORS, API Management policies, authentication, routing, Function authentication, the Function itself, AI Search, or the LLM. Keeping the path shorter makes debugging much easier.

Recommended Implementation Order

Step	Action
1	Connect React to the local Function now. Send POST requests to <code>http://localhost:7071/api/policy-chat</code> and replace the temporary/mock adapter response with the real Function response.
2	Finish and test the complete RAG application locally. Test multiple questions, conversationId handling, loading indicators, error handling, citations/source metadata, empty requests, and Function/Search/LLM failures.
3	Deploy the Azure Function to Azure. Move from the localhost Function endpoint to the cloud-hosted Azure Function endpoint.
4	Test React against the cloud Function endpoint. Confirm the same end-to-end behavior works when the Function is running in Azure.

Step	Action
5	Add Azure API Management. Use it as the API Gateway for authentication, rate limiting, API policies, logging, versioning, and controlled exposure of the Function API.

Target Cloud Architecture

```

React Policy Client
  | HTTPS
  v
Azure API Management (API Gateway)
  | - Authentication
  | - Rate limiting
  | - Policies
  | - Logging
  | - API versioning
  v
Azure Function App (RAG Orchestration)
  |
  +--> Azure AI Search
  +--> Foundry / Azure-hosted LLM
  |
  v
RAG Response

```

Project Milestone Sequence

Stage	What You Are Proving	Status / Recommendation
1	Search retrieves policy chunks	Completed
2	Function accepts policy queries	Completed
3	Function orchestrates Search + LLM	Essentially completed
4	React communicates with Function	DO THIS NOW
5	Complete end-to-end RAG UI	Next
6	Deploy Function to Azure	After local testing
7	React to cloud Function testing	After deployment
8	Add Azure API Management	Then
9	Authentication / authorization	Then
10	Monitoring, security, production hardening	Later

Recommended Next Development Task

Connect the working ChatGPT-style React client to
<http://localhost:7071/api/policy-chat>, replacing the temporary/mock
ChatModelAdapter response with the actual Policy Function response.

This creates the first genuinely complete version of the Policy Assistant application. Once that path is stable, deploy the Function and then introduce API Management as a separate infrastructure milestone.